Backend Audit and Prioritized Plan

Prepared 2026-08-19. Scope: `main/backend` (built with Node.js and Express, using the Firebase Admin SDK to talk to the database). This is an internal company tool with roughly 100 to 500 people using it daily, expected to grow toward 1,000 or more. It is not a large-scale, public SaaS product, and the recommendations below are sized accordingly. No microservices, Kafka, Redis, or GraphQL (all more complex, large-scale infrastructure tools) are being proposed here; they would be overkill for a tool this size.

Do not start implementing anything from this document without first confirming the priority order: fix everything marked P0 first, then P1, and so on.

---

1. Current Architecture

- **The technology stack:** Express 4 (the web server framework), the Firebase Admin SDK (for both Firestore, the database, and Firebase Auth, the login system), JWT (JSON Web Tokens, a way of signing a secure login token) for app sessions, Nodemailer over SMTP for sending emails, and `express-async-errors`, a small library that makes sure errors thrown inside async code get caught centrally instead of crashing silently. It runs either as a normal, long-running Node.js process, or as a Vercel serverless function (the code in `server.js` skips starting its own listener when it detects it's running on Vercel).
- **How requests flow through the code:** a request hits a file in `routes/`, passes through `authMiddleware` (confirms who's making the request), then `roleMiddleware` or `requirePermission` (confirms they're allowed to do this), then reaches a file in `controllers/`, which talks to Firestore directly. There's no separate service or repository layer in between.
- **Login:** Firebase Auth is the one source of truth for passwords. When someone logs in, the backend re-checks the password against Firebase's own Identity Toolkit service, then signs its own JWT (a token that expires after 7 days and carries the user's ID, email, role, name, and session ID inside it). `authMiddleware` then re-checks the account against a cache that refreshes every 60 seconds, plus a separate cache tracking revoked sessions that refreshes every 30 seconds, so a role change, a deactivated account, or a forced logout takes effect within about a minute.
- **Permissions:** there are three layers. `roleMiddleware` does a coarse check (is this role even allowed near this feature at all), `requirePermission` does a finer-grained check against a per-role action list stored in Firestore (currently only wired up for the IT asset management routes), and a few individual controllers also check ownership inline (does this specific record actually belong to this specific user). The available roles are: employee, hr, it, coordinator, founder, and superadmin.
- Session tracking (through a `sessions` collection) and audit logging (through an `audit_logs` collection, a record of who did what) are both real, working features, wired into every sensitive action a Super Admin can take.

2. What the API Can Do (by route file)

- `authRoutes` (`/api/auth`): register (open to anyone), log in (open to anyone), fetch your own profile, and re-verify your password.
- `hrRoutes` and `itRoutes`: create, read, update, and delete complaints, plus status and field updates, all restricted by role. IT additionally has full asset management, protected by the finer-grained permission check too.
- `founderRoutes`: a merged view of all complaints, user management, audit logs, analytics (with CSV export), search, an activity timeline, dashboard layout and overview, SLA policies, notification rules, role and action permissions, and department management. Nearly everything here is restricted to superadmin only, aside from a handful of read-only views open to any logged-in user.
- `securityRoutes` (`/api/founder/security`): active sessions, force-logout, failed login attempts, and account unlocking, all restricted to superadmin only.
- `approvalRoutes`, `leaveRoutes`, `coordinatorRoutes`, and `renderRoutes`: role-restricted create/read/update/delete plus approval-style decision endpoints for each of their respective areas.
- `hrDeskRoutes`: sending email, plus a generic create/read/update/delete setup shared across employees, candidates, interviews, meetings, attendance, feedback, and job postings, restricted to HR and founder roles only.

There are no automated tests anywhere in `main/backend` at this time.

3. Problems, in Priority Order

P0: Critical

Nothing new was found here. There is a known issue, already flagged separately in `INFRASTRUCTURE_PLAN.md`, involving `docs/login-credentials.pdf` (a file containing real employee passwords that is not yet tracked by Git). The action needed is simply to make sure it's never committed to the project's history, and to delete it from disk once it's no longer needed there.

P1: High Priority

#	PROBLEM	WHERE	SUGGESTED FIX
1	List views cap results at 200 records, but do so **without first sorting by date**. Because of how Firestore (the database) works, this means it returns an arbitrary set of 200 records, not necessarily the 200 most recent ones. Once any collection passes 200 total records, brand-new tickets, tasks, or leave requests can start silently disappearing from queues without anyone noticing.	`hrController.js:144,152`, `itController.js:150,158`, `founderController.js:62-64`, `approvalController.js:35`, `leaveController.js:46`, `assetController.js:41`, `renderController.js:7`, `taskProjectController.js:9,15`, `hrDeskController.js:34,48`	Add a proper "sort by date, newest first" step before applying the 200-record cap, along with any extra database indexes Firestore requires to support that sort.
2	The dashboard and analytics pages run **completely unrestricted** reads across seven entire collections at once: users, HR complaints, IT complaints, approvals, leave requests, assets, and failed login attempts. This scales with the total amount of data that has ever existed, not with what's actually shown on screen. This is the next likely place a usage-limit or cost problem could hit as the amount of data grows (the same category of problem that's already caused an incident once before).	`founderController.js:322-328` (the `computeAnalytics` function), `founderController.js:581-593` (the `getDashboardOverview` function)	Limit these to a specific date range, and/or use database-level counting features, instead of reading every single record.
3	The short-term caches used elsewhere (in `authMiddleware.js` and `sessions.js`) assume the server keeps running continuously in memory. But this app can also run as a serverless function on Vercel, where each "cold start" begins with a completely empty cache. That means the caching approach that fixed a previous usage-limit problem might not actually hold up on that kind of deployment.	`authMiddleware.js:16`, `utils/sessions.js:26`	First confirm which deployment setup is actually being used. If it's serverless, the caching strategy needs to be rethought. **Status as of 2026-08-19: blocked.** The deployment target hasn't been confirmed yet, so no code change was made in this pass.
4	Anyone can create a real account and log in as a regular employee, with no restrictions at all: no approval step, no check that the	`authRoutes.js:7`, `authController.js:27-62`	Either require an admin to approve new accounts, or at minimum add

#	PROBLEM	WHERE	SUGGESTED FIX
	email belongs to the company's own domain, and no limit on how many accounts can be created in a row.		a limit on repeated signup attempts (see item 6 below) and/or restrict signups to company email addresses.

P2: Medium Priority

#	PROBLEM	WHERE	SUGGESTED FIX
5	Text that users type in (a ticket's name and department, and the body of HR-desk emails) gets inserted directly into HTML emails without being escaped first. This means someone could type something designed to break or manipulate how the email renders.	`utils/mailer.js:36-39`, `hrDeskController.js:13`	Escape any user-typed text before it gets inserted into an HTML email.
6	There's no limit anywhere in the app on how many times someone can attempt an action in a row. Logging in has a per-account lockout after repeated failures, but there's no broader limit based on where a request is coming from, for example on the registration page. This was already flagged separately in `INFRASTRUCTURE_PLAN.md`.	the whole app	Add a rate-limiting library (`express-rate-limit`) to slow down repeated attempts from the same source.
7	When something unexpected goes wrong, the app's general error handler sends the raw internal error message straight back to whoever made the request. This is a minor risk of leaking internal details that shouldn't be visible outside the system.	`server.js:52-57`	Return a generic, safe message for unexpected errors. Only pass the actual message through for errors the code deliberately threw on purpose with a clear status and message attached.
8	The HR-desk feature's generic create/read/update/delete function has no list of which fields are actually allowed to be edited, unlike the equivalent pattern used everywhere else in the codebase. It currently accepts and saves whatever fields are sent in the request body, without checking them.	`hrDeskController.js:44-84`	Add an explicit list of which fields can be edited, matching the pattern already used throughout the rest of the codebase.
9	There are no automated tests anywhere. Important logic, like checking who owns a record, routing rules based on department, and status updates that need to happen together as one unit, has nothing catching it if a future change accidentally breaks it.	not applicable	A handful of tests covering `authMiddleware`, `roleMiddleware`, and the transactional (all-or-nothing) update paths would catch regressions cheaply and early.
10	The document `docs/BACKEND_WORKFLOW.md` is out of date. It says most of the interface still isn't connected to the backend, but the project's history shows Tickets, Approvals, Leave, Assets, Tasks,	documentation only	Update the document to reflect current reality, or delete it so it doesn't get mistaken for accurate, current information.

#	PROBLEM	WHERE	SUGGESTED FIX
	Rendering, and HR are all fully working now.		

P3: Low Priority

- The minimum password length is 6 characters, and that minimum is only actually enforced when an admin resets someone's password (`founderController.js:262`). Self-registration relies entirely on Firebase Auth's own built-in default. This is acceptable at the current scale of this tool; noting it here for visibility, not as something urgent to fix.
- `LOCK_THRESHOLD = 5` (the number of failed login attempts before an account locks) is a hardcoded number in the code (`authController.js:10`). Its own comment explains this was a deliberate choice, so this isn't a problem, just noted for completeness.

---

4. What's Already Fine, Don't Re-Fix These

- CORS (the setting that controls which websites are allowed to call this API) is properly restricted to a specific allow-list with a sensible cache duration, not left wide open.
- Updates that need to happen together as a single unit, like linking a status change to its approval record, correctly use Firestore's transaction feature, which re-reads the latest data before writing, in `itController` , `hrController` , and `approvalController` .
- There's no way for someone to grant themselves extra privileges. Self-registration always results in a plain employee account, nobody can mint themselves a founder or superadmin account, and a Super Admin can't accidentally lock themselves out.
- Session revocation and force-logout are both real, working features that take effect quickly thanks to the session ID plus the short-lived cache.
- Account lockout after repeated failed logins works correctly, and correctly resets after a successful login.
- Asset creation has a safeguard against malicious database document IDs being injected (`assetController.js:11`).
- The function that looks up user roles for a batch of records (`enrichWithUserRole`) correctly avoids the "one extra request per item" trap by batching and deduplicating its lookups.
- Every sensitive action a Super Admin can take is properly recorded in the audit log.
- The `.env` file (where secrets are stored locally) is correctly excluded from Git, and `config/firebase.js` fails gracefully by falling back to a local test database instead of crashing when real credentials aren't available.
- CSV exports are properly protected against CSV/formula injection, a trick where malicious spreadsheet formulas get smuggled into exported data.

Next Step

Confirm the priority order above, then start with the P1 items. Item 1 (adding the missing sort-by-date step) is the smallest change with the highest real-world impact: tickets silently vanishing from a queue is a bug that actively breaks support work, not just a theoretical risk.